SENTINEL

Worker threads, a shared queue and the track picture

Prelab 3

Status and deadline

This prelab is **required**, and it has **no marks**. Submit it on Brightspace by **Wednesday, November 18, 2026, by midnight**. The Lab 3 session is on Friday, November 20, 2026.

The oral exam of Lab 3[1] can ask you each question of this prelab. The deliverable of Lab 3 is the code that these questions describe: the Lab 3 Brief[2] states it. The course marks the deliverable out of 80, and it gives 4% of the course grade. The course marks the oral exam out of 20, and it gives 2%. Together they give 6%.

Instructions

In Lab 3, the centre is one process with many threads. The main thread puts each report into a shared queue, and a pool of worker threads takes the reports out and applies them to the track picture, the shared record of what is out there. Nothing protects the shared data in Lab 3. Each question covers one part of that work.

- Do this prelab alone, and write the answers in your own words.
- Each question names the manual pages that you need. If they are not on your machine, install them with `sudo apt install manpages manpages-dev`.
- You do not need the code of Lab 3. Part A names the lectures and the files that the questions use, and each other fact is in the question.
- You can run a program to check an answer. Give your answer first. Compile a program with threads with `gcc -pthread`.

Part A. How to write the code of Lab 3

Lab 3 asks for two kinds of code: the start and the stop of threads, and a queue. Learn each one here, before the session. `ipc.h`[3] gives the steps of each function, and each starter file in `src/libipc/` repeats them above the function.

Threads. Lecture 4.5 of the course, Threads, shows each call that Lab 3 uses: `pthread_create()`, `pthread_join()`, and the link with `-lpthread`. (The given `Makefile` compiles and links with `-pthread`, which does the same.) Its program `threadNumber.c` starts N threads, gives each thread the address of its OWN number, `&thread_numbers[i]`, and then joins each thread. Your

[1] The oral exam of Lab 3 is in the Lab 4 session. The Submission Guide, Section 3, `docs/submit/submit.pdf`, describes its format.

[2] The Lab 3 Brief, `docs/lab3/lab3_brief.pdf` in your repository. It states the contract of Lab 3, its ideas and its hand-in.

[3] `src/libipc/include/sentinel/ipc.h`, in your repository. It states what each function of Lab 3 does, in order, with each of its errors and its log lines.

`pool_create_deadline` and `pool_stop` do the same with the workers. Read `threadNumber.c`, then `man 3 pthread_create` and `man 3 pthread_join`.

The queue. Lecture 3.3, Semaphores, shows a *bounded buffer*, `prodCons.c`: an array with a fixed size, which a producer fills and a consumer empties. The queue of Lab 3 is a bounded buffer too, with three differences. Do not copy the lecture code:

- The lecture buffer is a stack: the last item in is the first out, with `count` only. The Lab 3 queue is first in, first out: a *ring* with `head` and `tail`.
- The lecture buffer uses a mutex and two semaphores. Lab 3 has no lock and no semaphore: Lab 3 shows what goes wrong without them, and Lab 4 adds them.
- The lecture producer waits when the buffer is full. A push to a full Lab 3 queue is refused at once, with `ENOSPC`.

In a ring, after the last slot an index goes back to slot 0. The operator `%` does it:

```
tail = (tail + 1) % capacity;    /* capacity 4: 0, 1, 2, 3, 0, 1, ... */
```

`head` moves in the same way. Question 3 follows a ring of 4 slots step by step.

Q1. What the threads of one process share

Read `man 7 pthreads`, `man 3 pthread_create`, `man 3 pthread_join` and `man 2 fork`. The centre of Lab 3 is one process with a main thread and 8 worker threads.

(a) For each item of this table, write *shared* or *its own*: is it shared by the threads of one process, or does each thread have its own? Then do the same for a parent and its child after `fork()`, as in Lab 2: write *shared*, *a copy* (the child gets a copy of the value of the parent) or *its own* (the child gets a new value).

Item	**Threads of one process**	**Parent and child after `fork()`**
A global variable, such as `long total;`		
A local variable of a function		
The heap, the memory of `malloc()`		
The open file descriptors		
The process ID		

(b) A worker thread returns from its start function, and no thread calls `pthread_join()` for it. What does the system keep for that thread? Why must the stop of the pool call `pthread_join()` for each worker before it frees the memory of the pool?

Q2. The argument of a thread

Read the description of `arg` in `man 3 pthread_create`. Look at this program:

```
static void *show(void *arg)
{
    int n = *(int *)arg;
```

```
    printf("worker %d\n", n);
    return NULL;
}

int main(void)
{
    pthread_t t[4];
    for (int i = 0; i < 4; i++) {
        pthread_create(&t[i], NULL, show, &i);
    }
    for (int k = 0; k < 4; k++) {
        pthread_join(t[k], NULL);
    }
    return 0;
}
```

The main thread can finish its loop before a new thread runs.

(a) The program is meant to print `worker 0` to `worker 3`, one time each. Why can it print another set of numbers, such as `worker 2` two times? Which numbers can a thread print? Why is the result not the same in each run?

(b) Change the program so that each thread prints its own number. Lecture 4.5 of the course, Threads, shows one way in its program `threadNumber.c`. The pool of Lab 3 gives each worker the address of its own block, `&workers[i]`. How does that do the same?

Q3. A ring buffer of 4 slots

The queue of Lab 3 is a ring buffer. It has `capacity` slots, and three numbers: `head`, the slot of the next report to take out; `tail`, the next free slot; and `count`, the number of reports that wait. At the start, all three are 0.

- A push: when `count` is equal to `capacity`, the push is refused. If not, the report goes into slot `tail`, then `tail = (tail + 1) % capacity`, and `count` increases by 1.
- A pop: when `count` is 0, the pop gives nothing. If not, it gives the report in slot `head`, then `head = (head + 1) % capacity`, and `count` decreases by 1.

(a) The capacity is 4. Do these nine operations in order: push A, push B, push C, pop, push D, push E, push F, pop, pop. Give `head`, `tail` and `count` after each operation. Which push is refused? Which report does each pop give?

(b) `head` and `tail` can be equal when the queue is empty, and also when it is full. For each case, give one point of (a), or the start. Why does the queue keep `count`?

Q4. A run that disagrees

Read Section 3.5 of the Lab 3 Brief[4] and the steps of `queue_pop` in `ipc.h`.[5] A run of 8 workers on 8 processors, with no lock, ended with these lines:

```
processed         20722 of 20705 queued
picture counted   20722 reports, 36 tracks (36 walked)
  accounted for                  20722   DOES NOT BALANCE -- nothing here is protected (R4)
shared data DISAGREES: unprotected workers lost or repeated updates (Part 4 fixes this)
```

(a) The workers processed 17 reports more than the centre queued. How can two workers take the same report? Which line of `queue_pop` copies the report, and why can both workers run it before either one changes `head`?

(b) In another run, the demo printed `queue says 18446744073709112535 reports still wait`, and the workers did not stop. Which field of the queue went below 0? Why does a `size_t` below 0 give a number this large? Why does the drain then never end?

[4] The Lab 3 Brief, `docs/lab3/lab3_brief.pdf` in your repository. It states the contract of Lab 3, its ideas and its hand-in.

[5] `src/libipc/include/sentinel/ipc.h`, in your repository. It states what each function of Lab 3 does, in order, with each of its errors and its log lines.